

Memoization

Memoization is the technique of remembering the return values from a function for the specific arguments supplied to it. For example, if someone asked me to find the square root of 720, which is the number that when multiplied by itself results in 720, I'd have to sit down with pencil and paper for a few minutes (or call `Math.sqrt(720)` in JavaScript or `math.sqrt(720)` in Python) to figure it out: 26.832815729997478. If they asked me again a few seconds later, I wouldn't have to repeat my calculation because I'd already have the answer at hand. By caching previously calculated results, memoization makes a trade-off to save on execution time by increasing memory usage.

Confusing *memoization* with *memorization* is a modern mistake made by many. (Feel free to make a memo to remind yourself of the difference.)

Top-Down Dynamic Programming

Memoization is a common strategy in *dynamic programming*, a computer programming technique that involves breaking a large problem into overlapping subproblems. This might sound a lot like the ordinary recursion we've already seen. The key difference is that dynamic programming uses recursion with repeated recursive cases; these are the *overlapping* subproblems.

For example, let's consider the recursive Fibonacci algorithm from Chapter 2. Making a recursive `fibonacci(6)` function call will in turn call `fibonacci(5)` and `fibonacci(4)`. Next, `fibonacci(5)` will call `fibonacci(4)` and `fibonacci(3)`. The subproblems of the Fibonacci algorithm overlap, because the `fibonacci(4)` call, and many others, are repeated. This makes generating Fibonacci numbers a dynamic programming problem.

An inefficiency exists here: performing those same calculations multiple times is unnecessary, because `fibonacci(4)` will always return the same thing, the integer 3. Instead, our program could just remember that if the argument to our recursive function is 4, the function should immediately return 3.